

TOKEN MANAGEMENT SYSTEM

Presentation Script + Viva Preparation Guide

Companion to: 'Blue and White Gradient Modern Project Presentation.pptx' (12 slides)

SHWETHIN	TEJASSH	STEVE IPE
NIKHIL STALIN	NOEL DENNY	TALAHA

How to use this document: each slide below has three parts - **SAY** (the spoken script), **KNOW** (facts you must be able to state without reading), and **Q&A** (questions evaluators are likely to ask, with answers). Read Section 0 first - your deck has 3 items that must be fixed before you present.

Stack: React + Vite frontend | Node.js + Express REST API | MongoDB Atlas (cloud) | JWT auth + bcrypt hashing | Deployed on Vercel

Demo account (seeded into Atlas on first connect): organizer@test.com / password123

0. URGENT - FIX THESE 3 THINGS IN THE PPTX BEFORE PRESENTING

The deck was made before the final code change. The project is now **pure MongoDB Atlas** - **all fallback code was deleted** (no mongodb-memory-server, no in-memory store). If a slide says otherwise and the evaluator checks the code, you lose credibility. Fix these:

1	Slide 5 (Technologies)	Remove the line 'mongodb-memory-server - Embedded database fallback' from the 'Other Technologies' box. Replace with: 'MongoDB Atlas - cloud-hosted database (only database used)'.
2	Slide 11 (Conclusion)	Remove the bullet 'Fallback storage makes the system suitable for demonstrations.' Replace with: 'Demo data is auto-seeded into Atlas on the first connection, so the demo works immediately.'
3	Slide 9 (Contributions)	MEMBER 1's description is still lorem ipsum placeholder text ('id eiusmod ex adipisicing...'). Replace with 1-2 real sentences.

Also on Slide 8: fix the typo 'Delete even' to 'Delete event'.

Timing plan (about 10-12 min): Slides 1-3: 2 min | Slides 4-5: 2 min | Slides 6-8: 3 min | Slide 9: 1.5 min (each member speaks) | Slides 10-12: 2.5 min + live demo.

SLIDE 1 - TITLE (Web Programming - Token Management System)

What the slide shows: Project title, course (Web Programming), and the six team members.

SAY (spoken script)

Good morning. We are presenting the Token Management System - a web application that replaces paper-based queue tokens with digital tokens issued by scanning a QR code.

An organizer - like a clinic or an event desk - creates an event and gets a QR code. Attendees simply scan it with their phone, type their name, and instantly receive a unique token number. No app install, no account.

It is a full-stack project: a React frontend, a Node.js and Express REST API, and MongoDB Atlas as a live cloud database. It is deployed on Vercel.

KNOW THIS COLD

- The one-line pitch: 'QR-code based digital queue token system - attendees get tokens without signing up.'
- The three tiers: React client (browser) -> Express API (Node) -> MongoDB Atlas (cloud DB).
- Both user types: organizer (logs in, manages queue) vs attendee (no login, just scans and joins).
- Demo credentials: organizer@test.com / password123 - seeded automatically into Atlas on first connect.
- Be able to name every team member and their contribution area (slide 9).

LIKELY VIVA Q&A

Q: What does your project do in one sentence?

A: It lets organizers run a queue where attendees join by scanning a QR code and instantly receive a unique, server-generated token number - no paper, no app, no account.

Q: Is this a real deployment or a localhost demo?

A: Both - it runs locally (Vite dev server + local Express on port 4000) and is deployed on Vercel with MongoDB Atlas as the production database.

Q: Why is this a 'web programming' project?

A: It exercises the full web stack: HTML/CSS/JS frontend, client-side routing, REST API design, middleware, authentication, database modeling, and cloud deployment.

SLIDE 2 - PROBLEM STATEMENT

What the slide shows: Six pain points of traditional queues: manual/paper tokens, long waits, hard large-queue management, allocation errors, duplicate numbers during simultaneous registration, no remote join.

SAY (spoken script)

Queues are everywhere - clinics, college fests, bank counters. The traditional way is a register book and chits of paper, and that breaks in predictable ways: handwriting errors, lost tokens, and no queue visibility.

The worst failure is at rush hour: two people registered at the same moment can end up with the same number. And there is no way to join a queue before physically standing in it. Our project attacks exactly these six problems.

KNOW THIS COLD

- Memorize the 6 problems and map each to our fix (they will ask 'how do you solve point 5?').
- Duplicate numbers under simultaneous registration -> solved by MongoDB's atomic \$inc operator.
- No remote joining -> solved by the QR code + public join URL - works from any phone browser.
- Manual errors / lost paper -> all data persisted in Atlas; organizer sees a live table.
- Every problem is solved by a concrete feature - be ready to point at the code for each.

LIKELY VIVA Q&A

Q: Which of these problems is the hardest to solve technically, and how do you solve it?

A: Duplicate token numbers during simultaneous registrations. We solve it with an atomic database operation. `Event.findByIdAndUpdate(id, { $inc: { lastTokenNumber: 1 } })` is executed as ONE indivisible operation by the MongoDB engine, so two requests can never read the same counter value.

Q: Couldn't you just print tokens from a spreadsheet?

A: A spreadsheet still needs a human at the counter and gives attendees no self-service. Our system lets attendees issue their own token from their phone, and the organizer only manages the queue state.

Q: Who are the two user roles in your system, and why does only one need an account?

A: Organizer (creates/manages events - needs an account because queue management must be restricted to the owner) and attendee (joins via QR - an account would add friction with zero benefit since they only take one token).

Q: What prevents a random person from messing up someone else's queue?

A: They can only take a token from a public link - which is the intended behavior of an open queue. Everything destructive (viewing the token list, marking done, editing, deleting) sits behind JWT authentication.

SLIDE 3 - PROPOSED SOLUTION (QR-Based Digital Token System)

What the slide shows: The 3-step flow: (1) organizer creates event and QR is generated, (2) attendee scans QR and enters name, (3) attendee receives unique token; organizer manages queue.

SAY (spoken script)

Our solution is a three-step flow. Step one: the organizer creates an event - the app immediately renders a QR code and a shareable link on the manage page.

Step two: an attendee scans that QR code. The link opens a public join page in any phone browser - no login - where they type their name and tap 'Get My Token'.

Step three: the server atomically assigns the next number in that event's sequence and returns the token, which appears as a big badge on the attendee's phone. The organizer's table shows them in arrival order, and one click marks each person done.

KNOW THIS COLD

- The exact 3 steps in order - you will be asked to walk through them.
- The QR encodes ONLY a URL: `window.location.origin + '/join/' + '?title='`. No server call is needed to create the QR - it is rendered client-side by the `QRCodeSVG` component from `qrcode.react`.
- `window.location.origin` is why the same code works on localhost, on a laptop's Wi-Fi IP for phone demos, and on the Vercel domain - zero config.
- `eventId` in the URL is the event's MongoDB `_id`.
- The `?title=` parameter is only a cosmetic fallback so the page header shows even if the title fetch fails.

LIKELY VIVA Q&A

Q: Why a QR code and not just a short link / SMS?

A: QR needs zero typing and zero setup - a phone camera is now a universal QR scanner. A link can also be shared (we show it next to the QR), and SMS would need a paid gateway and phone-number collection.

Q: Why can attendees join without an account? Isn't that insecure?

A: Joining a queue is a deliberately public action, like taking a paper token. The API splits routes: `/api/public` is open, while `/api/events` and `/api/tokens` are locked behind JWT verification.

Q: What actually happens server-side when someone scans and joins?

A: The app calls `POST /api/public/events/:id/join` with the name; the server atomically increments the event's `lastTokenNumber` with `$inc`, creates a `Token` document with that number and status `'waiting'`, and returns it with `HTTP 201`.

Q: If two people scan at the exact same time, what happens?

A: MongoDB serializes updates per document, so the two `$inc` operations execute one after the other - the first gets number 5, the second number 6. Duplicates are impossible by construction.

SLIDE 4 - SYSTEM FLOW / ARCHITECTURE (diagram slide)

What the slide shows: This slide has no extractable text - it is likely the architecture/flow diagram. Talk through the boxes as below.

SAY (spoken script)

Here is the full architecture. The browser runs our React single-page application - five screens served by Vite in development and by Vercel's CDN in production.

Every data action goes from the browser to our REST API. In development the Vite dev server proxies /api requests to Express on port 4000; in production Vercel rewrites /api requests to a serverless function, api/index.js, which runs the same Express app.

The API layer is pure Node and Express with middleware for CORS, JSON parsing, and JWT verification. All data lives in MongoDB Atlas - users, events, and tokens - accessed through three Mongoose models.

KNOW THIS COLD

- 3-tier architecture: presentation tier (React SPA), application tier (Express REST API), data tier (MongoDB Atlas).
- Request path dev: React -> fetch('/api/...') -> Vite proxy -> Express (localhost:4000) -> Mongoose -> Atlas.
- Request path prod: React -> fetch('/api/...') -> Vercel rewrite -> api/index.js (serverless) -> same Express app -> Atlas.
- api/index.js calls connectDB() before every request but reuses the warm connection (readyState check) - only cold starts actually reconnect.
- There is NO local database, NO in-memory store, NO localStorage data - Atlas is the single source of truth. If Atlas is unreachable, the API returns HTTP 500 immediately (fail-fast, no fallback).
- JWT rides in the Authorization: Bearer header on protected requests.

LIKELY VIVA Q&A

Q: Explain your architecture diagram.

A: Three tiers: the React frontend in the browser; the Express REST API - locally on :4000, on Vercel as a serverless function; and MongoDB Atlas as the only database. The client talks to the API exclusively through /api/* fetch calls with a JWT attached for protected routes.

Q: How is production different from development in this diagram?

A: Dev uses the Vite proxy to reach a long-running Express process; production replaces that with Vercel rewrites hitting api/index.js, a serverless wrapper that connects to Atlas on cold starts and reuses the connection on warm invocations.

Q: What happens if the database is down?

A: By design there is no fallback. connectDB() throws, and the serverless function responds 500 'Database connection failed'. We chose fail-fast so misconfiguration (wrong MONGO_URI, missing IP whitelist) is visible immediately instead of silently serving stale data.

Q: Where does authentication happen in this flow?

A: The JWT is issued by POST /api/auth/login, stored in the browser's localStorage, and verified in a middleware that runs before every /api/events and /api/tokens route.

SLIDE 5 - TECHNOLOGIES USED

What the slide shows: Frontend (React.js, React Router, Vite, JavaScript, localStorage for JWT), Backend (Node.js, Express, Mongoose, bcryptjs, jsonwebtoken), Others (MongoDB, CORS, dotenv). NOTE: delete 'mongodb-memory-server' line - see Section 0.

SAY (spoken script)

Our stack is the MERN pattern with Vite. On the frontend: React for the component-based UI, React Router for page navigation, Vite as the dev server and bundler, and plain JavaScript throughout.

On the backend: Node.js as the runtime, Express for the REST API, Mongoose to model MongoDB documents, bcryptjs to hash passwords, and jsonwebtoken for stateless authentication.

The database is MongoDB Atlas - the managed cloud version - and CORS plus dotenv handle cross-origin access and environment configuration. One important clarification: browser localStorage stores ONLY the JWT login session - every piece of application data lives in Atlas.

KNOW THIS COLD

- One-line purpose of EVERY technology on this slide (they will pick any and ask 'why this?').
- localStorage holds only 'jwt' and a small session cache - never users/events/tokens data.
- Vite is used (not Create React App) because it is fast (esbuild, native ESM) and has a built-in dev proxy for /api.
- bcryptjs is pure JS (no native compile step) - chosen over bcrypt for easy installs.
- CORS: needed in production where the Vite proxy doesn't exist; app.use(cors()) accepts cross-origin calls.
- dotenv loads MONGO_URI/JWT_SECRET from server/.env locally; on Vercel, env vars come from the dashboard and the dotenv load is wrapped in try/catch so it is optional.
- (If asked about mongodb-memory-server: it was removed - the project is now pure Atlas with no fallback.)

LIKELY VIVA Q&A

Q: Why React instead of plain HTML/JavaScript?

A: Component reuse (5 pages share layout/forms/buttons), declarative state-driven rendering, and client-side routing give an SPA feel without page reloads - while staying simple with plain useState/useEffect hooks.

Q: Why Vite over Create React App?

A: Much faster cold start and rebuilds via native ES modules, smaller config, and a first-class dev-server proxy that forwards /api to Express without CORS issues.

Q: Why Mongoose instead of the raw MongoDB driver?

A: Schema validation (required, unique, enum, defaults), a clean model API (find, findOneAndUpdate, save), and ObjectId refs - structure the raw driver doesn't give, at zero extra infra cost.

Q: What do bcryptjs and jsonwebtoken each do?

A: bcryptjs one-way hashes passwords with a per-user salt (factor 10) so the DB never stores plain passwords; jsonwebtoken signs a stateless credential ({ userId }, 1-day expiry) that proves identity on protected routes.

Q: Why is CORS needed if the frontend and API are on the same Vercel domain?

A: Locally the Vite proxy makes requests same-origin, but in production the browser can reach the API via different hosts/preview URLs; cors() guarantees acceptance in both environments.

Q: What is dotenv for, and why is it in a try/catch?

A: It loads server/.env (MONGO_URI, JWT_SECRET) in local development. Vercel injects env vars itself and server/.env doesn't exist there, so an unconditional require could crash the function - the try/catch makes it optional.

SLIDE 6 - REACT IMPLEMENTATION

What the slide shows: Main components: App.jsx (routes), Login.jsx, Register.jsx, Dashboard.jsx, ManageEvent.jsx, JoinEvent.jsx, and storage.js (API communication + JWT handling).

SAY (spoken script)

The React app is a five-page SPA. App.jsx defines the routes: /login and /register for organizers, /dashboard for the event list, /events/:id for the manage screen with the QR code and token table, and /join/:id - the public attendee page.

Every page talks to the backend through one service file, storage.js. Despite the name, it stores nothing - it is an API client. Every function is a fetch call to /api, and it automatically attaches the JWT from localStorage as a Bearer token.

State is deliberately simple: local useState per page, useEffect for data loading, and no Redux or Context - the pages don't share complex state.

KNOW THIS COLD

- All 6 components + what each one does (they'll ask you to open any page and explain it).
- Routes: '/' redirects to /dashboard; /events/:id and /join/:id use URL params (useParams).
- storage.js internals: apiFetch() sets Content-Type, adds Authorization: Bearer from localStorage 'jwt', parses JSON, and throws Error(data.message) on non-2xx.
- getCurrentUser() decodes the JWT payload client-side just to know if a session exists - real security is server-side.
- Auth-guard pattern in Dashboard/ManageEvent: if no session -> navigate('/login').
- QR is rendered with QRCodeSVG from qrcode.react; the join page uses useSearchParams for the ?title= fallback.
- Forms are controlled components (useState bound to inputs).

LIKELY VIVA Q&A

Q: Why function components with hooks instead of class components or Redux?

A: State is page-local and small - events, tokens, form fields. useState/useEffect cover that with far less boilerplate than classes or a global store, keeping every file easy to trace in a viva.

Q: What does storage.js actually do? The name suggests localStorage.

A: Historical name - it is an API client. Each method (login, getEvents, createEvent, joinEvent, markTokenDone...) issues a fetch to the corresponding /api/* endpoint; localStorage is touched only to read/write the 'jwt' key.

Q: How does the frontend protect /dashboard?

A: Two layers: a UX guard - getCurrentUser() returns null without a JWT, so the page redirects to /login - and real enforcement server-side, where the API returns 401 without a valid token, so the page has no data to show anyway.

Q: How does the QR code get the right URL on a phone over Wi-Fi?

A: The URL is built from window.location.origin. If the organizer opens the app via http://192.168.1.5:5173, the QR encodes that same origin, so any phone on the network opens a working link.

Q: How do you show the token number after joining?

A: joinEvent POSTs the name, the server returns the created token document, the page does setMyToken(data), and conditional rendering swaps the form for a badge showing #tokenNumber and status.

SLIDE 7 - NODE.JS & EXPRESS.JS IMPLEMENTATION

What the slide shows: Server responsibilities: start Express, connect DB, CORS+JSON middleware, mount routes, /api/health. NOTE: the 'database fallback modes' line is outdated - there are no fallbacks now (see Section 0).

SAY (spoken script)

The backend entry point, server.js, does five things: it loads environment variables, configures the Express app with CORS and JSON body parsing, mounts the four route files, exposes a /api/health endpoint, and manages the MongoDB Atlas connection.

Route mounting is where security is enforced: /api/auth and /api/public are open, while /api/events and /api/tokens are mounted behind our JWT middleware.

On Vercel, the same app runs through api/index.js, a small serverless wrapper that ensures the database is connected and then passes the request into Express. If Atlas can't be reached, the connection throws and the API honestly answers HTTP 500 - there is no fake fallback data.

KNOW THIS COLD

- connectDB(): skips if already connected (readyState === 1), throws if MONGO_URI missing, connects with 5s serverSelectionTimeoutMS, logs 'Connected to MongoDB Atlas'.

- Demo seeding: after first connect, if `User.countDocuments() === 0`, it creates `organizer@test.com/password123`, one event 'Campus Tech Expo 2026', and 3 tokens. Seeding - NOT a fallback: it only runs after a successful Atlas connection, on an empty database, once.
- `/api/health` returns `{ status: 'ok', db: 'connected' | 'disconnected' }`.
- Middleware order: `cors()` -> `express.json()` -> routes. `express.json()` populates `req.body` for POST/PUT.
- `app.listen(PORT, '0.0.0.0')` for local dev; on Vercel there is no listener - the function is invoked per request.
- `JWT_SECRET` fallback chain: env var, else a demo default set in 3 files (`server.js`, auth routes, middleware).

LIKELY VIVA Q&A

Q: What middleware do you use and why?

A: `cors()` to accept cross-origin requests in production, `express.json()` to parse request bodies, and a custom JWT auth middleware mounted before `/api/events` and `/api/tokens`. Middleware runs in the order it's registered.

Q: How does your server work as a Vercel serverless function?

A: `api/index.js` requires the same Express app, awaits `connectDB()` - which no-ops on warm invocations thanks to the `readyState` check - then calls `app(req, res)`. No `app.listen`; Vercel invokes the exported handler per request.

Q: What does `/api/health` do and why keep it?

A: It reports liveness and database state - `{ status: 'ok', db: 'connected' }`. After deploying we hit it first; if db is 'disconnected' we know the `MONGO_URI` env var is wrong before touching the UI.

Q: What happens on Vercel if `MONGO_URI` is missing or wrong?

A: `connectDB` throws ('`MONGO_URI` not set' or a connection error), `api/index.js` catches it and returns 500 with the error message. The fix is to correct the env var and redeploy - visible failure beats silent wrong data.

Q: Why do you seed demo data inside `connectDB`? Isn't that a fallback?

A: No - it executes only AFTER a successful Atlas connection and only when the users collection is empty, so a fresh cluster instantly has a demoable account, one event and three tokens. It depends on Atlas; it cannot replace it.

Q: Why listen on 0.0.0.0 instead of localhost?

A: `0.0.0.0` binds all network interfaces so phones on the same Wi-Fi (and hosted preview environments) can reach the dev server; `localhost` would only accept connections from the same machine.

SLIDE 8 - DATABASE DESIGN & CRUD OPERATIONS

What the slide shows: Three models (User, Event, Token), event CRUD, token create/list/status-update, and the token lifecycle waiting -> done. (Fix typo: 'Delete even' -> 'Delete event'.)

SAY (spoken script)

We designed three Mongoose models. A User is an organizer account - name, unique email, and a bcrypt-hashed password. An Event belongs to one user through an owner ObjectId reference and carries lastTokenNumber, the counter for its queue. A Token references its event, has a sequential tokenNumber, the attendee name, and a status that moves from waiting to done, with automatic createdAt timestamps.

The organizer has full event CRUD: create with a title, read all their own events, update the title, and delete - which also cascade-deletes that event's tokens. Tokens are created publicly through the QR join, listed by event in arrival order, and updated to done from the manage table.

KNOW THIS COLD

- All three schemas, field by field (esp. Event: title, owner ref->User, lastTokenNumber default 0).
- Relationships via ObjectId + ref (Event.owner -> User; Token.event -> Event) - MongoDB has no foreign keys; we query by the id directly.
- Cascade delete: Token.deleteMany({ event: id }) right after deleting the event - no orphaned tokens.
- Atomicity: \$inc on lastTokenNumber is one engine-level operation - the concurrency guarantee.
- { new: true } returns the post-update document in findOneAndUpdate/findByIdAndUpdate.
- Ownership scoping: every protected query filters by owner: req.userId - you can only ever touch your own events.
- Validation happens in two layers: schema (required/unique/enum/default) + route checks (e.g. status must be 'waiting' or 'done' -> 400).

LIKELY VIVA Q&A

Q: Take me through your three collections and how they're linked.

A: users stores organizer accounts with hashed passwords. events stores queues with an owner field - an ObjectId referencing users._id - and a lastTokenNumber counter. tokens store queue entries with an event field referencing events._id, a tokenNumber, the attendee name, and a waiting/done status enum.

Q: Why a separate lastTokenNumber counter instead of counting tokens?

A: Counting is not safe under concurrency and breaks when tokens are deleted. A dedicated atomically-incremented counter gives a monotonic sequence per event - number 12 always means the 12th person who ever joined.

Q: What does \$inc do that a normal update can't?

A: Event.findByIdAndUpdate(id, { \$inc: { lastTokenNumber: 1 } }, { new: true }) performs read-modify-write inside the MongoDB engine as a single atomic step. Two simultaneous joins can never observe the same counter value - that is how duplicate token numbers are prevented.

Q: What happens to tokens when an event is deleted?

A: MongoDB has no ON DELETE CASCADE, so routes/events.js runs Token.deleteMany({ event: req.params.id }) immediately after Event.findOneAndDelete - manually guaranteeing referential integrity.

Q: How do you enforce that organizers only see their own events?

A: Every query in routes/events.js includes owner: req.userId in the filter, where req.userId comes from the verified JWT. Another organizer's event ID returns 404 - the database never even matches the document.

Q: How is the token status lifecycle modeled?

A: status is an enum ['waiting','done'] defaulting to 'waiting'. It flips via PUT /api/tokens/:id, which validates the value and rejects anything else with 400.

SLIDE 9 - INDIVIDUAL CONTRIBUTIONS (6 members)

What the slide shows: Six contribution slots. Fix MEMBER 1's lorem-ipsum text (Section 0). Assign the members to the slots below that match your deck's layout.

SAY (spoken script)

Here is how we divided the work. Each member owns one vertical slice and will answer for it today - member one: React routing plus login and registration; member two: the attendee join flow and the public token API; member three: the event backend and database operations; member four: the dashboard, event management UI and QR code; member five: JWT authentication and security; member six: the token backend, atomic token allocation and server setup.

(Each member should speak ~30-40 seconds on THEIR slice: what they built, what file(s), and one technical choice they made.)

KNOW THIS COLD

- Fill in your names against the 6 slots BEFORE presenting - evaluators ask each person about their own part.
- M1 files: client/src/App.jsx, pages/Login.jsx, pages/Register.jsx.
- M2 files: client/src/pages/JoinEvent.jsx, server/routes/public.js.
- M3 files: server/routes/events.js, server/models/Event.js (+ User.js/Token.js schemas).
- M4 files: client/src/pages/Dashboard.jsx, pages/ManageEvent.jsx (QR, token table).
- M5 files: server/routes/auth.js, server/middleware/auth.js, JWT_SECRET handling in storage.js.
- M6 files: server/routes/tokens.js, server/server.js (connectDB, seeding), api/index.js, vercel.json.
- Evaluator trick: they ask a member about ANOTHER member's part - everyone should know the whole flow at a high level.

LIKELY VIVA Q&A

Q: M1 - How does routing know which page to show?

A: react-router-dom matches the URL against the Routes declared in App.jsx - '/events/:id' captures the id segment via useParams, and '/' uses a Navigate redirect to '/dashboard'.

Q: M2 - Why is your join endpoint under /api/public?

A: Because attendees have no accounts; it's mounted without the auth middleware. It only exposes the event title and the join action - everything sensitive stays behind JWT.

Q: M2/M6 - Two phones join at the same moment - same token number?

A: No. Both hits execute \$inc as serialized single-document operations in the MongoDB engine, so one gets 5 and the next 6. Atomicity is the whole design.

Q: M3 - Show me the delete flow.

A: DELETE /api/events/:id runs Event.findOneAndDelete({ _id, owner: req.userId }) - 404 if not found or not mine - then Token.deleteMany({ event: id }) to cascade-remove its tokens.

Q: M4 - Where does the QR code come from? Is it an image from the server?

A: It's rendered entirely in the browser - QRCodeSVG from qrcode.react turns the join URL string into an SVG. No server request, no image file.

Q: M5 - What's inside your JWT, and where's the secret?

A: The payload carries only { userId } plus iat/exp (1-day), HMAC-SHA256-signed with JWT_SECRET from env vars. localStorage holds just that string; the middleware verifies it on every protected request.

Q: M6 - What did you set up for deployment?

A: vercel.json defines the build (client build + server install), the output dir, and rewrites sending /api/* to api/index.js and everything else to index.html; plus Atlas env vars and Network Access 0.0.0.0/0 so Vercel's dynamic IPs can reach the database.

SLIDE 10 - TESTING & RESULTS

What the slide shows: A test table: registration -> account created; login -> JWT generated; create event; QR URL; scan opens join page; join -> unique token; simultaneous joins -> unique numbers; view tokens; mark done; plus GET /api/health.

SAY (spoken script)

We tested the system as an end-to-end chain that mirrors a real event. Register creates the account with a hashed password; login returns a JWT; creating an event persists it; the QR encodes a working join URL; scanning opens the public join page; joining issues a unique token.

The critical test is concurrency: multiple simultaneous joins still produced unique, sequential token numbers - proving the atomic increment. Finally, viewing tokens shows the queue in arrival order, marking done updates the status, and /api/health confirms the service and database connection.

KNOW THIS COLD

- The full test chain in order and WHAT each test proves (not just what it does).
- How testing was done: api-tests.http (VS Code REST Client file with all 13 requests) + manual browser/phone flows.
- Negative tests you should mention proactively: empty name -> 400; wrong password -> 401; missing Authorization header -> 401; unknown event id -> 404; invalid status value -> 400.
- The concurrency test: two join requests fired together must return sequential numbers, e.g. 5 and 6 - that's the \$inc guarantee in action.
- /api/health returns { status:'ok', db:'connected' } - the first thing to check after any deploy.

LIKELY VIVA Q&A

Q: How exactly did you test the simultaneous-join scenario?

A: By firing multiple join requests at the same event at the same time (two phones/scripts together) and verifying the returned tokenNumbers were unique and sequential - which the atomic \$inc makes the only possible outcome.

Q: What negative/edge cases did you test?

A: Empty name on join (400), duplicate email registration (400), wrong credentials (401), no/expired JWT on protected routes (401), invalid ObjectId or foreign event id (404/500 path), and invalid token status (400).

Q: Is your testing manual or automated? How would you automate it?

A: Manual but scripted - every endpoint is encoded in api-tests.http so the whole chain runs in one click. To automate: Jest + Supertest against the Express app, or a Postman/Newman collection in CI.

Q: What does GET /api/health actually verify?

A: That the function is alive AND connected to Atlas - it reads mongoose.connection.readyState and reports db as 'connected' or 'disconnected'.

Q: What would you test next if you had more time?

A: Load testing many concurrent joins, expiry behavior of the 1-day JWT, and ownership isolation - proving organizer B can never read organizer A's events or tokens.

SLIDE 11 - CONCLUSION & FUTURE SCOPE

What the slide shows: Seven conclusion bullets (REPLACE the 'fallback storage' bullet - Section 0) and eight future-scope items.

SAY (spoken script)

To conclude: we built a working digital queue system around QR codes. Organizers create and control events; attendees join from any phone with no account; authentication is stateless JWT with bcrypt-hashed passwords; and all data - users, events, tokens - is persisted in MongoDB Atlas.

The one design decision we're proudest of is the atomic \$inc token allocator - uniqueness under concurrency is guaranteed by the database engine itself, not by our code's timing.

Looking forward, the top items are real-time updates over WebSockets or SSE, a live 'your position in line' view for attendees, notifications, pagination and search for big queues, role-based access, analytics, rate limiting, and refresh-token/password-reset support.

KNOW THIS COLD

- Every future item with ONE LINE on HOW you'd build it (evaluators pick one and ask).
- WebSockets/SSE: a channel per event that pushes new tokens to organizer + attendee screens instead of manual Refresh.
- Queue position: count waiting tokens with tokenNumber less than mine, return it with the join response.
- Rate limiting: a middleware (e.g., express-rate-limit) capping requests per IP on public routes.
- Role-based access: add role to the User schema and JWT payload; a requireRole middleware guards admin routes.
- Refresh tokens: short-lived access JWT + long-lived refresh JWT stored server-side, enabling revocation.
- Analytics: aggregate pipelines per event (tokens per hour, average wait = done time - createdAt).
- Known limitation to admit if asked: JWTs can't be revoked before expiry (stateless), and tokens.js doesn't yet verify that the token's event belongs to the requesting organizer - both listed in future scope.

LIKELY VIVA Q&A

Q: Pick one future-scope item - say WebSockets - and outline the implementation.

A: Keep the REST API as-is; add an SSE endpoint like GET /api/events/:id/stream that registers the response in an in-memory set keyed by event. Whenever join/mark-done commits, write the fresh token list to every registered stream - organizer and attendee screens update instantly with no polling.

Q: How would you implement role-based access control?

A: Add a role field (organizer/admin) to the User schema, include it in the JWT payload at login, attach it to req in the auth middleware, and add a requireAdmin check in front of sensitive routes - about 15 lines total.

Q: What is the single biggest weakness of the current system?

A: Honest answer: the tokens routes don't yet verify event ownership, so a logged-in organizer could mark another event's token done if they knew its id. Fix: load the token's event and compare owner to req.userId before updating - one query.

Q: Why not sessions instead of JWT? Would you switch?

A: JWT keeps the API stateless - perfect for serverless where any instance must verify any request. Switching to server sessions would need a session store (e.g., Redis) shared across function instances; JWT avoids that infrastructure entirely.

Q: If the queue gets to 10,000 tokens, what breaks first?

A: The manage page - fetching all tokens every refresh. That's why pagination/skip-limit and search are in future scope; the DB side is fine since token queries hit a single indexed event field.

SLIDE 12 - THANK YOU / LIVE DEMO & QUESTIONS

What the slide shows: Closing slide. Use this moment for the live demo and to invite questions.

SAY (spoken script)

Thank you. We'd be happy to demonstrate the system live now - we'll create an event, and if you scan this QR code with your phone, you'll get a real token from our database, and you can watch it appear on the manage

screen.

We welcome your questions.

KNOW THIS COLD

- Run the demo in this exact order to avoid fumbling: 1) login as organizer@test.com / password123, 2) open the seeded event's manage page, 3) show the QR + link, 4) scan w/ a phone (or open the link in an incognito tab), 5) join as a test name, 6) hit Refresh on the manage table - token appears, 7) Mark Done - status flips, 8) show /api/health JSON as the finale ('db: connected').
- Pre-demo checklist: backend running (or Vercel URL live), Atlas Network Access allows connection, phone on same Wi-Fi (local demo), Deployment Protection disabled (Vercel), browser logged out to show the full login flow.
- Keep these tabs open for code questions: server.js, routes/public.js (\$inc), middleware/auth.js, models/Event.js, api/index.js, api-tests.http.
- If Wi-Fi/phone fails mid-demo: fall back to an incognito browser tab on the same machine - identical flow.

LIKELY VIVA Q&A

Q: Show me where the token number actually comes from in the code.

A: Open routes/public.js: Event.findByIdAndUpdate with { \$inc: { lastTokenNumber: 1 } }, { new: true } - then new Token({ tokenNumber: event.lastTokenNumber ... }). The returned document's counter becomes the token number.

Q: Show me the security boundary in the code.

A: server.js route mounting: /api/events and /api/tokens pass through authMiddleware, which verifies the Bearer JWT and sets req.userId - every handler then filters by owner: req.userId.

Q: Show me the database - live. Where is this data right now?

A: MongoDB Atlas cluster - collections users, events, tokens. The join we just did inserted one document into tokens and incremented events.lastTokenNumber - visible in Atlas within seconds.

Q: What would break if I deleted MONGO_URI from Vercel right now?

A: The next cold function call would throw 'MONGO_URI not set' and the API would answer 500 'Database connection failed' - fail-fast by design; /api/health is where you'd see the flip to disconnected.

Q: Why should evaluators believe the concurrency claim without a load test?

A: Because uniqueness doesn't depend on our timing - \$inc is implemented inside the MongoDB storage engine as an atomic single-document update; the guarantee holds at any concurrency level by construction.

MASTER RAPID-FIRE SHEET (one breath before walking in)

Q: App in one line?

A: QR-based digital queue: scan, enter name, get a unique token - no account needed.

Q: Stack?

A: React+Vite SPA, Node/Express REST API, MongoDB Atlas - deployed on Vercel.

Q: Where's data stored?

A: Only MongoDB Atlas. localStorage holds just the JWT.

Q: Any fallback database?

A: No - removed deliberately. Atlas failure = honest HTTP 500.

Q: Passwords in DB?

A: Never plain - bcrypt hash, salt factor 10, compared at login.

Q: JWT contents?

A: { userId } + iat/exp (1 day), HMAC-SHA256 with JWT_SECRET.

Q: Why call it 'jwt' not 'token'?

A: Queue tickets are already called tokens - avoids name collision.

Q: Middleware?

A: cors, express.json, custom JWT auth on /api/events + /api/tokens.

Q: Duplicate token numbers?

A: Impossible - atomic per-document \$inc in the MongoDB engine.

Q: Event delete side effects?

A: Token.deleteMany cascades - no orphans.

Q: Ownership isolation?

A: Every query filters owner: req.userId from the verified JWT.

Q: Token lifecycle?

A: waiting (default, enum) -> done via validated PUT /api/tokens/:id.

Q: QR contents?

A: Just a URL: origin + /join/ + ?title=... rendered by QRCodeSVG client-side.

Q: Why public routes exist?

A: Attendees don't log in; only title-read and join are public.

Q: Health endpoint?

A: GET /api/health -> { status:'ok', db:'connected' }.

Q: Seeding vs fallback?

A: Seeding runs only AFTER a live Atlas connect on an EMPTY db, once - it needs Atlas.

Q: Vite proxy?

A: Dev-only: /api -> localhost:4000, so no CORS locally; production uses Vercel rewrites + cors().

Q: Biggest admitted weakness?

A: tokens.js lacks an ownership check (fix in future scope) and JWTs are irrevocable until expiry.

Good luck - own the atomic \$inc story, it's your strongest technical card. If the answers sound too polished, follow with the live demo; a working token appearing on the manage table wins every viva.